

Agentic Engineer Capstone Architecture Write-Up

1. Architecture Overview

This capstone implements a governed multi-agent engineering pipeline for AI-assisted code review and software-change validation.

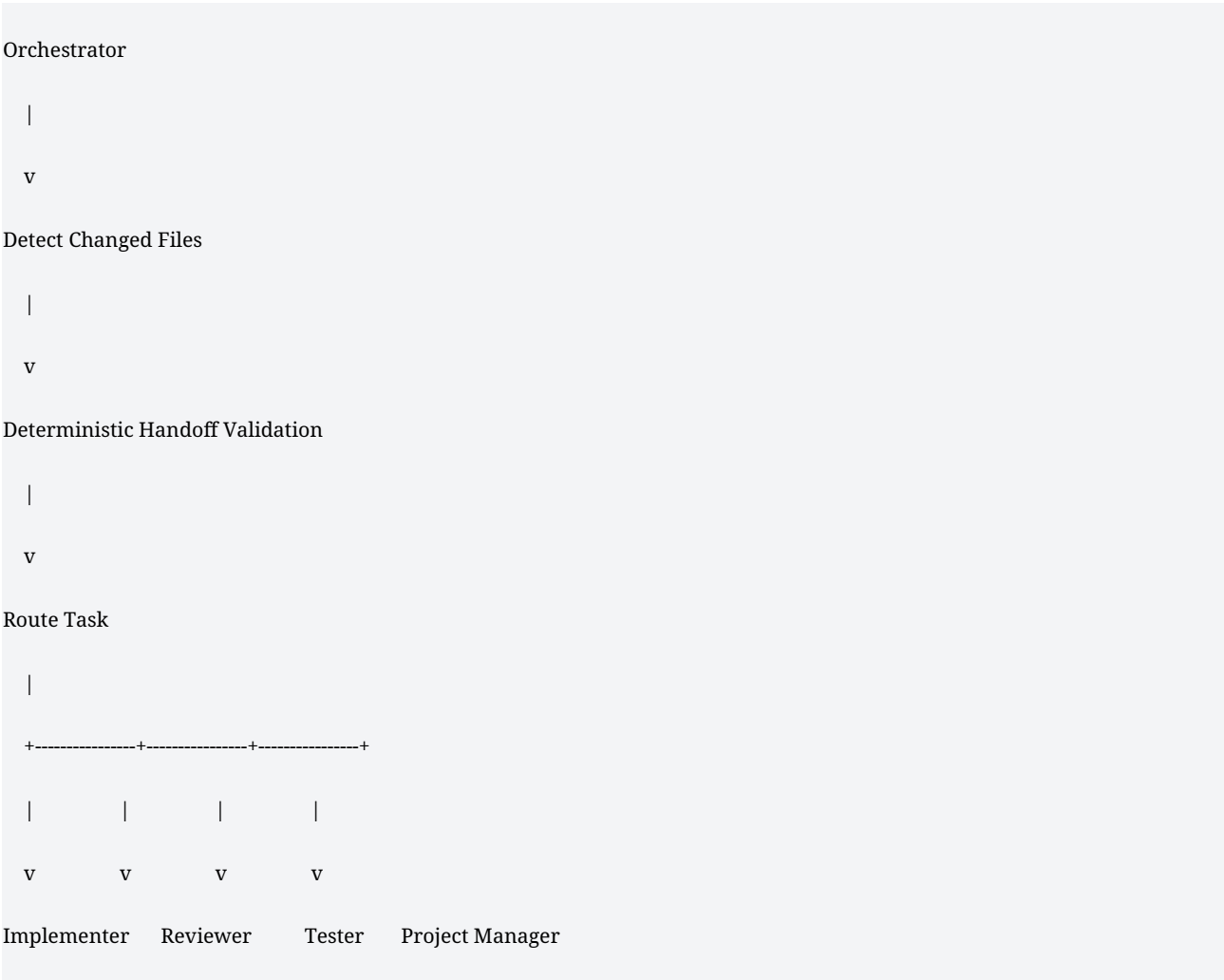
The system combines three execution approaches:

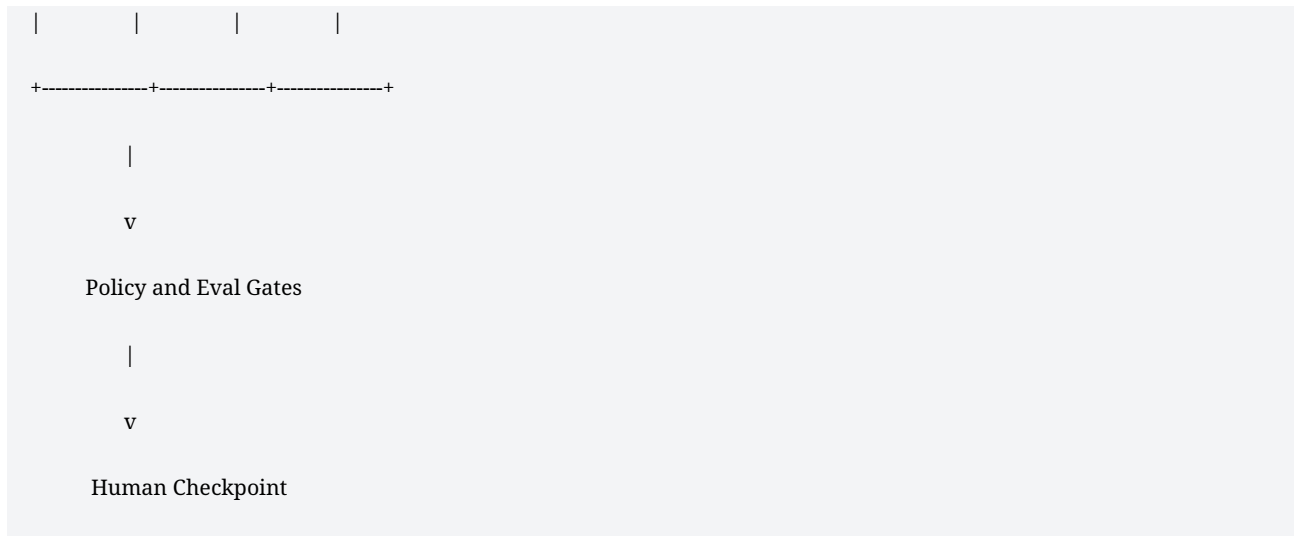
- **Agentic execution** for work that requires reasoning, interpretation, review, or synthesis.
- **Deterministic execution** for stable structural validation and routing logic where the same input should produce the same result.
- **Human checkpoints** for actions that require approval, elevated permissions, or consequential decisions.

The goal is not to make every workflow step agentic. Instead, each step is assigned to the simplest execution method that can perform it safely and reliably.

2. Workflow

The implemented orchestration flow is:





The orchestrator coordinates the workflow rather than giving every agent unrestricted access to the entire system.

Before agent delegation, the handoff structure is validated using deterministic code. Invalid or malformed handoffs can therefore be rejected before consuming model resources or reaching a specialized agent.

3. Orchestrator

The Orchestrator is the parent coordination role.

Its responsibilities include:

- coordinating the workflow;
- detecting or interpreting the work that needs to be performed;
- delegating tasks to the appropriate specialized role;
- maintaining workflow-level visibility;
- accessing audit information when required;
- enforcing the transition between workflow stages.

The Orchestrator has the broadest storage permissions because it owns workflow coordination and auditing.

Its configured storage operations include:

- write_entry
- read_entry
- list_entries
- update_entry
- delete_entry
- audit_read

The Orchestrator also has retrieval access with a data-classification ceiling of **confidential**.

This broader access is limited to the coordination role rather than being inherited by every subagent.

4. Deterministic Handoff Validation

One workflow step has been deliberately moved out of the agentic layer.

The handoff validator uses deterministic code rather than a language model.

The orchestration documentation identifies the validator as:

```
validate_handoff_deterministic.py
```

The validator checks the structure of the handoff before routing continues.
This step does not require interpretation or language judgment. A handoff either conforms to the expected structure or it does not.

Using deterministic validation provides:

- repeatable behavior;
- lower latency;
- no model-token cost for the validation itself;
- simpler testing;
- clearer audit evidence;
- predictable failure handling.

The validator requires no MCP access.

This conversion demonstrates the project's right-tool principle: a stable structural operation should not remain agentic simply because an agent is capable of performing it.

5. Routing

After validation, the workflow routes work according to task type.

The current routing design includes:

Task Type	Responsible Role
-----	-----
Code or implementation work	Implementer
Code-review work	Reviewer
Test execution and reporting	Tester
Description or project summary work	Project Manager

A sample deterministic router is also included in the repository as a second deterministic decision-history example.

Routing therefore has a documented path toward deterministic execution when task categories are sufficiently stable and specifiable.

6. Implementer

The Implementer performs code and implementation work.

Storage access

The Implementer may:

- write entries;
- read entries;
- list entries;

- update entries.

It may **not delete entries**.

The deletion restriction exists because implementation work does not require removal of stored project state, and an over-broad delete grant was identified as a governance risk.

Retrieval

Retrieval is permitted with a maximum data-classification level of:

```
internal
```

Skills

The Implementer may activate:

- run-tests
- summarize-session

It may not activate:

- draft-pr-description

PR-description ownership belongs to the Project Manager.

Autonomy

The Implementer operates at **medium autonomy**.

Human review or escalation is required before:

- shell commands outside the normal test suite;
- writing outside the current feature-branch scope.

Its container uses:

```
workspace: read-write
```

```
memory: mounted
```

7. Reviewer

The Reviewer is intentionally read-only and advisory.

Its purpose is to inspect work independently rather than modify the work it is evaluating.

Storage access

The Reviewer may:

- read entries;
- list entries.

It may not:

- write entries;
- update entries;
- delete entries;

- inspect audit records.

Retrieval

Retrieval is permitted up to the **internal** classification level.

Skills

The Reviewer may use:

- summarize-session

It may not use:

- run-tests
- draft-pr-description

The governance policy explicitly prevents the Reviewer from running tools that would change workspace state.

Autonomy

Reviewer autonomy is **low**.

The Reviewer's output is advisory. Any action beyond producing review findings must be escalated to the Orchestrator.

Its container uses:

workspace: read-only

memory: omitted

This separation protects reviewer independence and prevents a reviewing agent from silently changing the implementation it is supposed to inspect.

8. Tester

The Tester executes or evaluates tests and reports the results.

Storage access

The Tester may:

- read entries;
- list entries.

It may not:

- write entries;
- update entries;
- delete entries;
- read audit records.

Retrieval

The Tester may retrieve supporting information up to the **internal** classification level.

Skills

The Tester may activate:

- run-tests
- summarize-session

It may not activate:

- draft-pr-description

Autonomy

Tester autonomy is **low**.

Any attempt to write to tracked repository files or stored project state requires escalation.

Its workspace remains read-only, and persistent memory is omitted.

This design separates test execution and reporting from implementation authority.

9. Project Manager

The Project Manager handles descriptions and project-level summaries rather than implementation or testing.

Storage access

The Project Manager may:

- read entries;
- list entries.

It may not:

- write entries;
- update entries;
- delete entries;
- inspect audit information.

Retrieval

Retrieval is denied for this role in the current workflow.

Its recorded data-classification ceiling is therefore **public**.

Skills

The Project Manager owns:

draft-pr-description

It may also:

summarize-session

It may not run tests.

This separation prevents presentation and summary responsibilities from acquiring unrelated engineering capabilities.

10. MCP Boundaries

The architecture uses MCP-backed storage and retrieval with role-specific access.

MCP access is not globally exposed.

Instead, operations are granted according to role responsibilities.

Examples include:

- the Orchestrator receiving coordination and audit permissions;
- the Implementer receiving state-update permissions without delete access;
- the Reviewer and Tester receiving read-oriented access;
- the Project Manager receiving read access but no retrieval capability;
- deterministic validation requiring no MCP access.

This structure follows least privilege.

A role receives only the operations necessary for its responsibility, and anything not explicitly granted is denied.

11. Data Classification

The governance design includes explicit data-classification ceilings.

Current ceilings include:

Role	Maximum Classification
-----	-----
Orchestrator	Confidential
Implementer	Internal
Reviewer	Internal
Tester	Internal
Project Manager	Public

This prevents a role from gaining access to higher-sensitivity data simply because the underlying MCP service is technically capable of returning it.

12. Persistent Storage and Retrieval

Persistent storage supports workflow state and engineering decision records.

Retrieval provides relevant reference context for roles that require it.

Retrieval is permitted for:

- Orchestrator
- Implementer
- Reviewer

- Tester

Retrieval is denied for:

- Project Manager

The repository also includes:

- retrieval ground-truth evidence;
- retrieval validation evidence;
- a retrieval quality report.

These artifacts allow retrieval behavior to be evaluated rather than assumed to work correctly.

13. Governance Model

The governance model begins with a deny-by-default rule:

Every role starts with no access.

Permissions must then be explicitly granted and justified.

Access widening requires a pull request containing:

- the proposed permission;
- a concrete justification;
- confirmation that it does not conflict with known calibration near-misses.

Governance therefore exists in both documentation and enforceable configuration rather than relying only on agent instructions.

The repository contains policy tests, governed-file tests, enforcement verification, and red-team evidence that support this design.

14. Human Checkpoints

Human review remains part of the architecture.

The system does not assume that agent autonomy should replace accountable decisions.

Examples of checkpoint conditions include:

- Implementer shell activity outside the approved test path;
- writes outside the intended feature scope;
- Reviewer attempts to perform actions rather than advisory review;
- Tester attempts to modify tracked source files or stored project state;
- requests requiring permissions beyond the role's policy.

The workflow therefore escalates instead of automatically expanding an agent's authority.

15. Evaluation and Policy Gates

Specialized-agent results pass through policy and evaluation gates before the workflow is considered complete.

The repository includes evaluation assets such as:

- holdout tasks;

- rubric-based evaluation;
- deterministic tests;
- regression tests;
- policy tests;
- governed-file tests;
- red-team prompts and results;
- enforcement verification;
- calibration records.

The purpose of these gates is to make workflow quality and policy compliance testable rather than relying on a model's self-assessment.

16. CI/CD Integration

The repository includes a GitHub Actions workflow at:

```
.github/workflows/ci.yml
```

The CI/CD layer is used to connect repository changes with automated evaluation and policy checking.

The architecture treats governance and evaluation as part of the engineering change process rather than as optional manual review performed after development.

Detailed CI design evidence is maintained separately in:

```
docs/ci-step-design.md
```

17. Right-Tool Architecture

The architecture deliberately combines agents, deterministic code, and humans.

Agentic steps

Agents are used when the work requires:

- interpretation;
- planning;
- implementation reasoning;
- code review;
- synthesis;
- context-dependent judgment.

Deterministic steps

Deterministic code is used for:

- handoff validation;
- stable structural checks;
- predictable routing where rules are sufficiently defined;
- governance and policy tests.

Human steps

Humans remain responsible when:

- permissions must be expanded;
- an agent exceeds its defined scope;
- consequential or elevated actions are requested;
- final approval is required.

This separation avoids using an expensive or nondeterministic agent for work that simpler code can perform reliably.

18. Auditability

The architecture is designed so that important decisions can be traced through repository artifacts.

Evidence includes:

- routing and tool-grant configuration;
- governance policy;
- calibration log;
- evaluation outputs;
- red-team results;
- CI results;
- deterministic tests;
- ADRs;
- human checkpoint rules.

This allows a reviewer to inspect not only what the system did, but also why a particular role was permitted or denied an action.

19. Security and Failure Containment

The architecture reduces the effect of agent failure through multiple boundaries:

- 1\ Role-specific tool grants
- 2\ Data-classification ceilings
- 3\ Read-only roles
- 4\ Container permission differences
- 5\ Deterministic input validation
- 6\ Evaluation gates
- 7\ Policy enforcement
- 8\ Human escalation
- 9\ Red-team and policy-bypass testing

A failure in one agent therefore does not automatically grant access to every tool, all stored data, or unrestricted repository modification.

20. Architecture Summary

The capstone architecture demonstrates a governed agentic engineering system rather than a collection of unrestricted AI agents.

The central design principles are:

- specialized agent roles;
- least-privilege access;
- deterministic validation where judgment is unnecessary;
- scoped MCP storage and retrieval;
- explicit data-classification boundaries;
- evaluation and governance gates;
- human escalation for higher-risk actions;
- auditable engineering decisions.

Together, these components create a pipeline in which AI agents can perform judgment-heavy engineering work while deterministic controls and human oversight constrain the actions that require stronger predictability or accountability.